

检查点 3：数据增广实现、可视化验证与开销分析

2026 年 4 月 17 日

1 实验背景

数据增广 (Data Augmentation) 的目标是在不增加额外标注成本的前提下，通过对已有样本施加变换来扩展训练分布，从而提升检测模型的泛化能力。对于目标检测任务而言，增广不仅改变图像外观，还会改变目标位置、尺度、遮挡关系和背景布局，因此需要同时关注图像变换效果与 bbox 同步变换正确性。

本阶段围绕以下三个目标展开：

1. 实现不少于 6 种图像数据增广方法，并接入统一的检测样本格式。
2. 可视化比较增广前后的图像与 bbox，验证标注是否同步更新。
3. 将增广加入 PyTorch Dataset 后测量额外开销，比较无增广与有增广时的加载速度差异。

2 任务 3.1：数据增广方法实现

2.1 方法选择

根据任务要求，本阶段实现了 6 种数据增广方法，其中 4 种为必做方法，2 种为补充方法：

1. RandomFlip
2. ColorJitter
3. MixUp
4. Mosaic
5. RandomAffine
6. RandomCrop

这些方法覆盖了三类常见检测增广场景：

- 外观变换：ColorJitter
- 单图几何变换：RandomFlip、RandomAffine、RandomCrop
- 多图组合变换：MixUp、Mosaic

2.2 实现原则

为了便于后续验证与测速，所有增广方法均建立在统一的检测样本格式上，输入输出都采用：

`image + boxes + labels`

其中图像为张量，边界框统一采用 `xyxy` 格式。几何类增广要求同步变换 `bbox`，多图增广要求正确合并不同样本的框与类别，并过滤越界框、无效框和面积过小框。

2.3 各方法实现思路

方法	主要作用	具体实现方式
RandomFlip	模拟左右方向变化	以一定概率对图像做水平翻转，同时按照图像宽度重新计算 <code>bbox</code> 的左右边界。
ColorJitter	模拟亮度、对比度和颜色变化	对图像像素做亮度、对比度、饱和度和色调扰动， <code>bbox</code> 保持不变。
RandomAffine	模拟平移、缩放、旋转和错切	对图像做仿射变换，并对每个 <code>bbox</code> 四个角点应用同样的几何映射，再重建外接矩形。
RandomCrop	模拟局部视野与目标截断	随机裁剪图像区域，重新计算裁剪后 <code>bbox</code> ，与裁剪区域无交集的框被丢弃。
MixUp	增加目标重叠与背景复杂度	将两张图像按权重融合，保留并合并两张图对应的 <code>bbox</code> 与 <code>labels</code> 。
Mosaic	增加小目标、密集目标和边界目标样本	将 4 张图拼接到同一画布，对每张图的 <code>bbox</code> 做缩放和平移，并对越界框裁剪。

表 1: 六种数据增广方法及其实现思路

2.4 代码组织

本阶段实现文件主要位于 `Unified/data_augmentation/scripts` 目录下：

- `detection_augmentations.py`: 实现 6 种增广方法与组合式增强流程。
- `detection_augmentation_dataset.py`: 定义训练用 `AugmentedDetectionDataset`，从统一 `manifest` 读取样本并接入增广。

3 任务 3.2：可视化检验增广前后图像与 `bbox`

3.1 验证目的

在检测任务中，增广是否正确并不只取决于图像视觉效果，还取决于边界框是否完成了同步更新。因此，本阶段为每种增广生成“增广前 / 增广后”对照图，并在图中绘制 `bbox`，用于

人工核查几何变换是否正确。

3.2 可视化结果



图 1: 六种数据增广的前后对照图及 bbox 可视化结果

3.3 验证结论

从可视化结果可以看出：

- RandomFlip 后，目标框随图像完成左右镜像，位置关系正确。
- ColorJitter 仅改变像素颜色，不影响 bbox。
- RandomAffine 和 RandomCrop 后，bbox 能跟随几何变换和裁剪区域重新计算。
- MixUp 能正确保留并合并两张图的目标框。
- Mosaic 能正确完成四张图拼接后的 bbox 缩放、平移和裁剪。

因此，这些增广不仅在视觉上生效，而且能够保持检测标注的一致性，为后续训练提供可用输入。

4 任务 3.3：增广加入 Dataset 后的计算开销分析

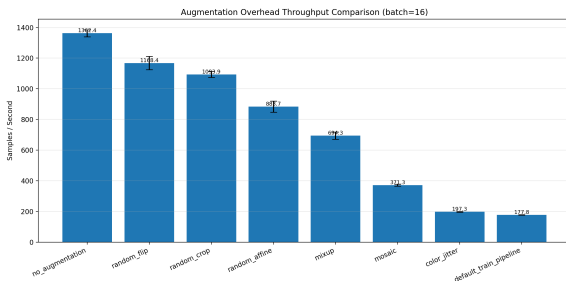
4.1 实验设置

为了公平比较不同增广带来的额外开销，本实验统一使用 PyTorch Dataset + DataLoader 链路，而不是单独调用增广函数。实验设置如下：

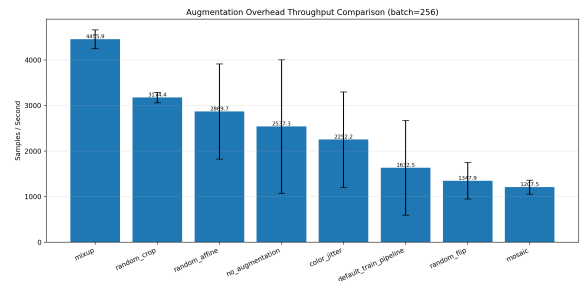
- 数据入口：Unified/manifests/all_val.jsonl
- 样本规模：1024
- 设备：cuda:0
- num_workers=8
- pin_memory=False
- persistent_workers=False
- prefetch_factor=4
- warm-up batches: 2
- repeats: 3
- batch size: 16 和 256

比较对象包括：no_augmentation、random_flip、color_jitter、random_affine、random_crop、mixup、mosaic 和 default_train_pipeline。

4.2 吞吐率对比图

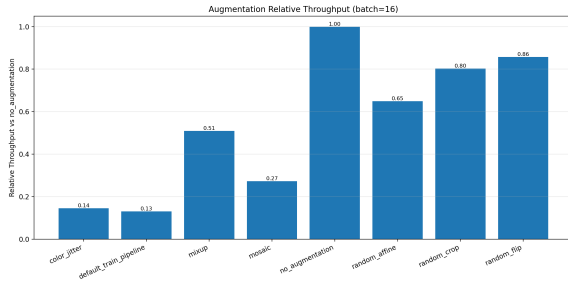


(a) batch size = 16

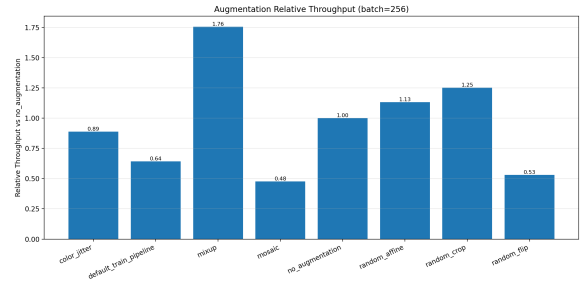


(b) batch size = 256

图 2: 不同增广策略下的数据加载吞吐率



(a) batch size = 16



(b) batch size = 256

图 3: 不同增广策略相对于无增广基线的吞吐率比例

4.3 关键结果

方法	batch=16 (samples/s)	batch=256 (samples/s)
no_augmentation	1362.42 $\pm$ 23.45	2537.25 $\pm$ 1467.22
random_flip	1168.39 $\pm$ 43.41	1347.88 $\pm$ 402.63
color_jitter	197.32 $\pm$ 2.03	2252.19 $\pm$ 1048.22
random_affine	883.72 $\pm$ 35.79	2869.69 $\pm$ 1044.44
random_crop	1093.88 $\pm$ 19.85	3174.37 $\pm$ 114.09
mixup	694.35 $\pm$ 23.49	4455.92 $\pm$ 209.06
mosaic	371.27 $\pm$ 5.91	1207.50 $\pm$ 153.57
default_train_pipeline	177.82 $\pm$ 1.39	1632.46 $\pm$ 1039.24

表 2: 不同增广策略的吞吐率比较

4.4 结果分析

- 在 batch size = 16 下，结果较稳定，能够较清晰地反映增广引入的额外开销。无增广基线达到 1362.42 samples/s。
- 轻量增广中，RandomFlip 和 RandomCrop 的吞吐率分别保留了基线的约 85.8% 和 80.3%，说明这两类变换的额外代价相对较小。
- RandomAffine 的吞吐率下降到基线的约 64.9%，说明几何映射和 bbox 重建带来了可观开销。
- MixUp 和 Mosaic 属于重增广。特别是 Mosaic 在 batch=16 下仅为 371.27 samples/s，约为无增广的 27.3%。
- 完整训练增强链路 default_train_pipeline 吞吐率最低，仅为 177.82 samples/s，说明多种增广叠加后会显著抬高数据准备成本。
- 在 batch size = 256 下，由于 1024 个样本只产生 2 个测量 batch，统计波动明显更大，因此这一组结果更适合作为大批量参考，而不宜单独作为严格排序依据。

- GPU 显存占用方面，Mosaic 的峰值最高，约为 301 MB，高于其他单项增广。

5 总结

本阶段完成了数据增广任务的三个核心部分：

1. 在统一检测样本格式上实现了 6 种数据增广方法，并接入训练用 Dataset。
2. 通过可视化结果验证了增广前后图像与 bbox 的同步关系，证明几何类与多图类增广都能正确更新标注。
3. 通过 PyTorch Dataset + DataLoader 测试了不同增广策略的额外开销，证明增广复杂度越高，数据加载吞吐率通常越低。

整体来看，RandomFlip 和 RandomCrop 适合作为代价较低的轻量增广；RandomAffine 具有中等开销；MixUp 和 Mosaic 虽然能增强样本复杂度，但会带来明显的额外计算负担。完整训练增广流水线则在提升样本多样性的同时显著降低了数据供给速度，因此在后续训练中需要结合模型收益与训练吞吐共同权衡。